

Vocabulary Dropout for Curriculum Diversity in LLM Co-Evolution

Jacob Dineen, Aswin RRV, Zhikun Xu, Ben Zhou
 Arizona State University
 {jdineen, aravik13, zhikunxu, xzhou202}@asu.edu

Abstract

Co-evolutionary self-play, where one language model generates problems and another solves them, promises curriculum learning without human supervision. The promise breaks down early in practice. The proposer converges to a narrow distribution of problems that satisfy the reward function, and the collapsed curriculum teaches the solver little, stalling the loop. We introduce vocabulary dropout, a lightweight intervention that randomly masks the proposer’s output logits during both policy training and curriculum generation. The mask is hard and non-stationary, so the proposer cannot lock into fixed token sequences. Training Qwen3-4B and Qwen3-8B on mathematical reasoning via R-Zero, vocabulary dropout sustains proposer diversity throughout training across lexical, semantic, and functional measures, and improves the solver by an average of +4.4 points at 8B with the largest gains on competition-level benchmarks. Explicit action-space constraints, filling the structural role that game rules fill in classical self-play, can keep co-evolution in language productive. Vocabulary dropout is one simple way to impose them.¹

1 Introduction

Self-play is a promising training paradigm in which models improve through competition or interaction with copies of themselves. This approach has been particularly successful in games, where the environment provides strong structural constraints on learning. In domains such as Go, StarCraft, and Dota, the rules of the game specify legal actions, fixed dynamics, and explicit objectives, making self-play well-defined (Tesauro, 1995). Beyond these constraints, methods such as population-based training, opponent sampling, and fictitious self-play (Vinyals et al., 2019; Berner et al., 2019; Heinrich & Silver, 2016) expose agents to a broader mixture of opponents and help reduce cycling or collapse to narrow strategies.

Recent co-evolutionary training frameworks apply a similar idea to language by splitting a single model into two roles, a *proposer* that generates reasoning problems and a *solver* that attempts them (Huang et al., 2026). The proposer is trained to produce problems at the edge of the solver’s ability, and the solver is trained on the resulting curriculum, creating a loop analogous to two-player self-play training. As the solver improves, the proposer gradually adapts, generating increasingly difficult problems to maintain the training signal. In practice, however, the proposer converges to a narrow set of question templates that satisfy the reward function, and the curriculum loses its value as a training signal (Liu et al., 2026; Chae et al., 2025; Pu et al., 2026). Our experiments show that this stagnation sets in early in self-play training, coinciding with the early plateau in solver capability reported in prior work (Huang et al., 2026). Standard diversity metrics fail to detect it.

We hypothesize that this collapse stems from a difference between games and language. Game environments are symbolically verifiable, so constructing a hard position generally requires finding one that is strategically demanding, and an opponent cannot stay challenging

¹Code available at https://github.com/ARC-ASU/vocab_dropout.

without actually improving. Natural-language questions offer no comparable verification. Neither the proposer nor the solver is bound by external rules here, and no environment certifies on its own that a question is difficult. This leaves the proposer able to satisfy the reward function without any grasp of why a question is hard. It settles on narrow templates that reliably induce solver errors or disagreement, and the curriculum ends up superficially varied while resting on a small set of underlying question templates.

Prior work attempts to mitigate this diversity collapse by anchoring the co-evolutionary loop to external signals (Zhao et al., 2025; Huang et al., 2026; Liu et al., 2025; Wilf et al., 2025), and Liu et al. (2026) argue more broadly that self-play stalls when the generated data ceases to provide learnable structure. These approaches improve stability but none directly constrain the proposer’s output space.

AlphaZero injects exploration noise scaled to the number of legal moves to prevent degenerate play (Silver et al., 2017), OpenAI Five randomizes game properties to force strategic diversity (Berner et al., 2019), and more broadly the structure of the action space shapes what policies can learn (Chandak et al., 2019; Farquhar et al., 2020; Sharma et al., 2026). We introduce **vocabulary dropout** (VD) to fill a structurally similar role in language generation. Where game rules constrain semantically meaningful actions, VD works one level down, on the token action space, as a hard non-stationary mask over the proposer’s output logits that removes a random subset of tokens each batch. A blocked token has probability zero rather than merely reduced probability, so optimization pressure cannot erode the constraint. And since the mask is redrawn every batch, the proposer can route around any particular masked token but has no way to pre-commit to a fixed substitution strategy (§4.1).

We evaluate vocabulary dropout within R-Zero (Huang et al., 2026), which rewards the proposer through solver self-consistency rather than an external verifier, training Qwen3-4B and Qwen3-8B base models via GRPO (Shao et al., 2024). Our contributions are threefold. We characterize proposer diversity dynamics across co-evolution iterations empirically, showing that functional stagnation sets in early and goes unnoticed by standard lexical metrics. We introduce vocabulary dropout as a simple hard constraint on the proposer’s action space, and show that it sustains diversity across lexical, semantic, and functional measures for the whole run. And we show that the sustained diversity improves the solver, by +4.4 points on average at 8B, with the largest gains on competition-level benchmarks.

2 Related work

Self-play and co-evolution for LLMs. A growing family of methods trains LLMs on curricula they generate themselves (RRV et al., 2026). SPIN refines an SFT model by having it distinguish human text from its own (Chen et al., 2024). Absolute Zero, R-Zero, MAE, and Socratic-Zero mine increasingly challenging tasks from minimal seed data, through executable environments or co-evolving challenger-solver pairs (Zhao et al., 2025; Huang et al., 2026; Chen et al., 2025; Wang et al., 2025a; Bailey et al., 2026). R-Few adds lightweight human supervision to stabilize self-evolution (Yu et al., 2025), and SPICE grounds generation in retrieved documents (Liu et al., 2025). Closest in spirit, Mishra (2026) counter curriculum collapse with a semantic coverage signal over an embedding-induced partition, though this too intervenes on problem selection and leaves the proposer’s action space untouched. We run our experiments in R-Zero’s two-model setup, where the challenger is directly observable and diversity interventions can be studied under control.

Mode collapse in LLM training. Policy-gradient methods concentrate probability on high-reward behaviors, causing entropy collapse even when rewards are verifiable (Zhou et al., 2025; Gai et al., 2025). Cui et al. (2025) establish that downstream performance is bottlenecked by entropy exhaustion. This applies not only to solvers but also to proposers in co-evolutionary settings, where the proposer’s policy gradient training drives it toward a narrow set of high-reward problem templates. RL-trained reasoning models exhibit reduced solution diversity (Yue et al., 2025; Karouzos et al., 2026) and may fail to develop reasoning behaviors absent from the base policy (RRV et al., 2025), while LLM populations grow homogeneous over generations (Jiang et al., 2025; Chae et al., 2025). Proposed mitigations

generally target the reward or sampling side, including entropy regularization (Cui et al., 2025), structured reward decomposition (Dineen et al., 2025; Ye et al., 2025), diversity-promoting rewards (Li et al., 2025), population-based training (Jaderberg et al., 2017), and inference-time noise injection (Khalid et al., 2026). Vocabulary dropout instead acts directly on the action space rather than the reward or sampling distribution.

Action-space design in RL. In classical RL, action representations shape exploration and generalization (Chandak et al., 2019; Farquhar et al., 2020). In games, rule-imposed constraints on legal moves are what make self-play productive (Silver et al., 2017; Berner et al., 2019). For language models, the vocabulary *is* the action space, and its size is typically fixed at the tokenizer’s full output. Vocabulary reduction has been studied for efficiency (Nozaki et al., 2025), and RLPT (Pang et al., 2026) masks contextually irrelevant tokens to concentrate the policy on promising outputs. Our vocabulary dropout inverts this logic, masking tokens randomly and non-stationarily to prevent concentration rather than encourage it.

3 Preliminaries: GRPO and R-Zero

R-Zero (Huang et al., 2026) trains a proposer π_P and a solver π_S for mathematical reasoning, both from one base checkpoint, alternating between the two roles (Figure 1, right). The optimizer in both phases is GRPO (Shao et al., 2024). GRPO avoids a learned value function altogether. It samples G responses $\{o_1, \dots, o_G\}$ from π_θ for a prompt x , gives each a reward r_i , and treats the group statistics as the baseline, $\hat{A}_i = (r_i - \mu_G)/\sigma_G$. The policy is then updated by maximizing the clipped surrogate:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \min(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon) \hat{A}_i) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \right] \quad (1)$$

with $\rho_i = \pi_\theta(o_i|x)/\pi_{\text{old}}(o_i|x)$ the importance ratio and a KL penalty holding π_θ near the reference model.

R-Zero rewards the proposer for problems that sit near the edge of what the solver can currently do. Each generated problem q goes to the frozen solver, which answers it M times, and the reward comes from how consistent those answers are with one another. Near-unanimous agreement signals a problem too easy to be useful. Heavy disagreement signals one that is too hard, or simply ill posed. Reward is highest at maximal solver uncertainty:

$$r_P(q) = \begin{cases} \min(\text{acc}(q), 1 - \text{acc}(q)) & \text{if } \text{acc}(q) \in [\tau_{\min}, \tau_{\max}] \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

where $\text{acc}(q)$ is the fraction of solver responses agreeing with the majority vote. R-Zero additionally penalizes intra-batch repetition via pairwise BLEU similarity, clustering near-duplicate questions and subtracting a penalty proportional to cluster size from the uncertainty reward, which encourages surface-level diversity within each batch. The solver trains on problems filtered from the proposer’s output, receiving binary reward for matching the majority-vote pseudo-label.

4 Method

4.1 Vocabulary dropout

Given the full vocabulary $\mathcal{V}$ with a small protected subset $\mathcal{F} \subset \mathcal{V}$ of format-critical tokens exempt from masking, such as the question tags and boxed-answer syntax (details in §B.2), we define a retention probability $\alpha \in (0, 1]$. At each batch b , we sample a mask $\mathbf{m}^{(b)} \in \{0, 1\}^{|\mathcal{V}|}$ where each entry is drawn independently:

$$m_v^{(b)} \sim \begin{cases} \text{Bernoulli}(\alpha) & \text{if } v \notin \mathcal{F} \\ 1 & \text{if } v \in \mathcal{F} \end{cases} \quad (3)$$

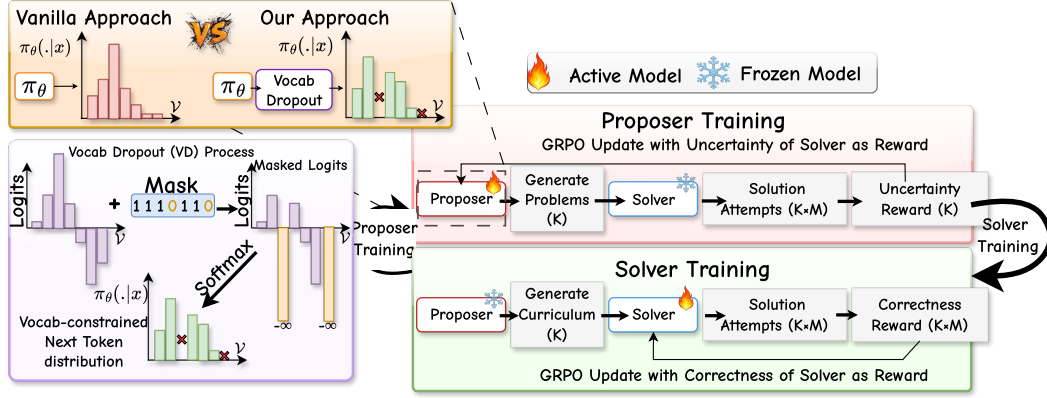


Figure 1: Training pipeline. **Left:** Vocabulary dropout masks a random subset of output logits, constraining the proposer’s token distribution. Orange denotes masked tokens. **Right:** The co-evolution loop. In **Phase 1** (proposer training), the proposer generates K problems, the frozen solver attempts each M times, and the proposer is rewarded based on solver uncertainty. In **Phase 2** (solver training), the frozen proposer generates a curriculum of K problems and the solver is rewarded for matching the correct answer.

The masked logits at batch b are then:

$$\tilde{\ell}_v^{(b)} = \begin{cases} \ell_v & \text{if } m_v^{(b)} = 1 \\ -\infty & \text{otherwise} \end{cases} \quad (4)$$

where ℓ_v is the original logit for token v . The resulting token distribution $\tilde{\pi}^{(b)}(\cdot|x) = \text{softmax}(\tilde{\ell}^{(b)})$ varies across batches even for identical inputs, since the surviving token set $\mathcal{V}^{(b)} = \{v : m_v^{(b)} = 1\}$ is resampled every batch.

The mask is applied in both phases of each iteration. During GRPO training, the mask is applied at the sampling stage of each rollout, so the policy can only generate from unmasked tokens. The subsequent policy-gradient update computes log-probabilities over the full vocabulary, allowing gradients to flow through all tokens. Because the available tokens change every batch, the policy cannot concentrate probability on a fixed set of high-reward sequences, which slows entropy collapse (Cui et al., 2025) and preserves exploration across the token distribution (Wang et al., 2025b). During curriculum generation, the per-batch mask likewise forces the proposer to produce varied phrasings and problem structures. The proposer therefore uses a larger effective vocabulary despite fewer tokens per batch, as confirmed in §6.2. Implementation details are in §B.2.

4.2 Training procedure

Both models are trained with GRPO (Shao et al., 2024) in alternating phases (pseudocode in Algorithm 1, prompt templates in §F). The proposer generates competition-level math problems via GRPO under vocabulary dropout with retention probability α . Each rollout is checked for valid format, and valid problems are scored by the frozen solver using the self-consistency reward from §3. After the proposer finishes training, it generates a large question set with vocabulary dropout still active. Problems within the difficulty window $([\tau_{\min}, \tau_{\max}] = [0.3, 0.7])$, following R-Zero) are retained, and the solver trains on this filtered set via GRPO with a binary reward for matching the proposer’s stated answer. This differs from R-Zero, which uses the solver’s majority-vote pseudo-label as the reward target.

5 Experiments

5.1 Setup

Vocabulary dropout. We apply vocabulary dropout in both phases (GRPO training and curriculum generation) and test retention probabilities $\alpha \in \{0.75, 0.85\}$ at each model scale, with $\alpha = 1.0$, the full vocabulary, as the baseline. Removing 15% or 25% of the non-protected vocabulary brackets the range where the tradeoff between diversity and output coherence shows up most clearly. Train-only and gen-only ablations isolate where the mask matters most.

Models and training. Base models are Qwen3-4B-Base and Qwen3-8B-Base (Yang et al., 2025), with the proposer and solver both initialized from the same checkpoint as in R-Zero (Huang et al., 2026). Each run takes 2 NVIDIA H200 GPUs and goes for 5 co-evolution iterations of GRPO (Shao et al., 2024) on ver1 (Sheng et al., 2025) with vLLM (Kwon et al., 2023) serving rollouts, longer than R-Zero’s 3-iteration protocol because peak evaluation performance did not always land on the final iteration there. Proposer GRPO samples $G=4$ responses per prompt at temperature 1.0 with a global batch size of 16, and proposer scoring queries the frozen solver $M=10$ times per question for self-consistency. The solver then trains on the filtered curriculum at global batch size 8. Optimization uses AdamW, learning rate 1×10^{-6} , weight decay 10^{-2} , and a KL penalty of $\beta=10^{-2}$, with full hyperparameters in SE.

We also train the two entropy-control methods of Cui et al. (2025), KL-Cov and Clip-Cov, which target the highest-covariance tokens in the proposer’s policy update and are applied to the proposer only (§B.4). Raising the proposer’s sampling temperature, the simplest intervention of all, breaks the pipeline before the first iteration completes, since at $T \geq 1.5$ the proposer cannot fill a batch with valid question-answer pairs.

Diversity metrics. We track proposer diversity at three granularities. Self-BLEU covers the lexical level, catching surface repetition. The semantic level uses two metrics, the Vendi score (Friedman & Dieng, 2023), an effective count of distinct question types in embedding space, and novelty rate, the fraction of questions whose nearest-neighbor cosine distance to all prior iterations exceeds 0.3, both on text-embedding-3-small (OpenAI, 2024) embeddings. For the functional level we compute epiplexity, which Finzi et al. (2026) introduced and Liu et al. (2026) applied to self-play curricula, measuring the learnable information content of generated questions through prequential minimum description length. Mean difficulty, proposer policy entropy (H_π), question length, and unique token counts are reported alongside (§§6.2, D.5).

Standard benchmarks. Solver capability is evaluated on GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), AMC (Mathematical Association of America, 2023), OlympiadBench (He et al., 2024), and AIME 2024/2025 (Mathematical Association of America, 2024), which together span grade-school arithmetic up to olympiad problems. Annealing experiments (§7) add Minerva Math (Lewkowycz et al., 2022). The untrained base models are evaluated under the same protocol as a reference point. Evaluation is zero-shot with the model’s default chat template (§F.2), temperature 0.7, and a maximum context of 4096 tokens, scored by exact match via math_verify (Hugging Face, 2025). Every number is an average over 3 independent runs with mean $\pm$ standard error (Miller, 2024).

6 Results

6.1 Downstream performance

Table 1 reports pass@1 accuracy at the final co-evolution iteration. We avoid selecting the best checkpoint or run post hoc to prevent selection bias (Cawley & Talbot, 2010; Dodge et al., 2019). Vocabulary dropout improves solver performance at both scales. We use $\text{VD}\langle\alpha\rangle$ as shorthand for vocabulary dropout at retention probability α (e.g., VD75 means $\alpha=0.75$). At

Table 1: Solver pass@1 accuracy (%) after 5 co-evolution iterations. Vocabulary dropout (VD) is applied to the *proposer* only, and the solver trains on the resulting curriculum. α controls the fraction of the vocabulary retained. Phase ablations decompose $\alpha=0.75$ into train-only and gen-only. Best per model in **green bold**.

Setting	Pass@1 (%)						Avg.
	MATH500	GSM8K	AMC	Olympiad	AIME'24	AIME'25	
Qwen3-4B							
Base (no training)	54.1±2.2	63.5±5.2	29.2±1.8	20.6±1.3	12.2±0.9	1.1±0.9	30.1
KL-Cov	60.8±1.9	82.6±2.0	42.0±3.8	23.0±0.8	9.3±2.6	4.0±1.1	37.0
Clip-Cov	65.8±1.1	85.4±1.0	39.5±3.4	24.5±0.2	12.7±1.7	4.0±1.7	38.7
Baseline (α=1.0)	64.2±0.8	85.2±0.7	41.7±1.4	23.2±0.3	8.9±0.9	6.7±1.6	38.3
α=0.85	65.7±0.9	81.8±0.4	50.0±2.4	24.7±0.9	8.9±1.8	4.4±0.9	39.3
α=0.75	66.0±0.7	82.4±1.6	35.8±5.9	24.8±0.1	5.6±1.8	4.4±1.8	36.5
train-only	64.5±0.4	85.4±1.1	31.7±1.8	23.6±0.2	11.1±3.3	7.8±1.8	37.4
gen-only	62.9±1.3	85.9±0.7	42.5±1.2	24.1±0.8	7.8±0.9	3.3±1.6	37.8
Qwen3-8B							
Base (no training)	70.7±0.1	82.2±0.7	45.8±3.6	18.1±1.3	8.9±2.4	7.8±0.9	38.9
KL-Cov	71.1±1.7	88.8±0.5	45.0±2.6	26.9±0.8	8.7±2.0	9.3±1.1	41.6
Clip-Cov	70.0±1.3	86.5±2.6	42.5±2.0	28.0±0.5	12.7±1.7	9.3±1.7	41.5
Baseline (α=1.0)	69.9±0.4	86.9±2.5	40.8±7.7	23.1±1.8	12.2±0.9	3.3±1.6	39.4
α=0.85	69.7±2.3	89.5±0.8	45.8±1.8	26.8±0.5	6.7±0.0	7.8±3.3	41.1
α=0.75	68.1±2.3	88.6±1.4	51.7±3.6	27.7±0.3	15.6±2.4	11.1±2.4	43.8
train-only	71.1±1.5	89.7±0.6	44.2±3.6	27.0±1.1	13.3±0.0	5.6±2.4	41.8
gen-only	71.0±2.6	87.0±1.2	49.2±4.1	29.2±0.7	11.1±0.9	5.6±2.4	42.2

8B, VD75 leads all configurations with a +4.4 point average improvement over the baseline, driven by strong gains on competition-level tasks (AMC, Olympiad, AIME). At 4B, VD85 improves over the baseline on average (39.3 vs. 38.3). All configurations, including the baseline, improve over the untrained base models. Both entropy-control methods improve over the no-regularization baseline at 8B (41.6 and 41.5 vs. 39.4 avg) but fall short of the best VD configuration at each scale. At 4B, KL-Cov falls below the baseline (37.0 vs. 38.3) while Clip-Cov roughly matches it (38.7).

On the proposer side, co-evolutionary training does not improve downstream reasoning ability (Table 7). The 8B proposer (VD75: 38.4, baseline: 38.8) slightly degrades relative to the base model (38.9), and 4B shows a similar pattern. At 4B, proposer performance drops modestly under dropout (VD85: 31.9 vs. baseline: 32.8), consistent with the tighter constraint on the smaller model. The proposer is optimized for calibrated problem generation rather than problem solving, and these objectives do not appear to transfer.

Ablations. Table 1 also isolates the effects of masking strength and dropout phase. At 4B, VD85 outperforms the baseline while VD75 underperforms, so the optimal masking strength is scale-dependent and an overly aggressive mask can exceed a small model’s capacity to compensate. The phase decomposition at 4B shows gen-only slightly edging train-only (37.8 vs. 37.4), but neither recovers the baseline, because what binds at this scale is masking strength rather than dropout phase.

At 8B the pattern flips. Stronger masking consistently improves performance, both single-phase variants surpass the baseline (42.2 and 41.8 vs. 39.4), and the model absorbs the full mask with no loss in output quality. Gen-only edges train-only here too, and the combination outperforms either alone. What separates the scales is whether the model has enough capacity to absorb VD75 at all. Embedding-level analysis of these phase ablations (§D.2, Table 9) confirms that gen-only contributes the larger share of diversity, but the combination sustains the strongest growth across iterations.

6.2 Proposer diversity

Baseline stagnation is early and hidden. Nearly all of the baseline proposer’s functional collapse happens between iterations 1 and 2, after which the metrics plateau (Figure 3). In a single iteration, mean difficulty (panel c) jumps from 0.50 to 0.86 at 4B and from 0.71 to 0.92 at 8B, past the $\tau_{\max}=0.7$ threshold on solver accuracy. Policy entropy (a), Vendi score (d), and epiplexity (f) lock in just as early. Self-BLEU and novelty rate (b, e), on the other hand, stay flat from start to finish, so the proposer is recycling similar questions in a way surface metrics simply miss. Diversity interventions applied after the first iteration may accordingly be too late to head off curriculum collapse.

Dropout sustains diversity across all tiers. Under dropout, all functional metrics continue to evolve past iteration 2, the proposer’s distribution remains responsive to the solver’s improvement rather than freezing after the first update, and every diversity metric we measure improves (Figure 3). At the *lexical* level, Self-BLEU is roughly $2\times$ lower under dropout at both scales (b). At the *semantic* level, the Vendi score shows a gap of ~ 15 effective question types at both scales (d), mean pairwise cosine distance with Qwen3-Embedding-4B (Li et al., 2025) is $+4\text{--}6\%$ higher under dropout, and NLI-based logical diversity (Stasaski & Hearst, 2022) is $+19\text{--}68\%$ higher (Table 8). Novelty rate is $\sim 1.5\text{--}2\times$ higher under dropout (e). At the *functional* level, epiplexity is $\sim 35\%$ higher under dropout (f), indicating substantially more learnable structure in the curriculum (computation details in §E.3, E.4). These rankings are robust to an encoder swap and unidirectional NLI scoring (§D.1). Cumulative Vendi scores (§D.1) show the same pattern, with the baseline plateauing after iteration 2–3 while VD75 continues to grow, holding $\sim 35\%$ higher cumulative diversity at both scales.

Figure 2 confirms this at the structural level. Dropout questions contain more distinct numeric values and are substantially longer, while baseline runs produce roughly $2\times$ more valid questions per iteration, consistent with simpler problems that pass format and reward filters at a higher rate. Representative question pairs in §C illustrate the contrast. At the token level, VD75 proposers use $36\text{--}52\%$ more unique tokens than the baseline (3,697 vs. 2,717 at 4B, 2,868 vs. 1,881 at 8B) despite fewer being available at any given step (§D.5). The non-stationary mask forces broader vocabulary use rather than concentration. KL-Cov and Clip-Cov stay within 2% of baseline on all four metrics at both scales, leaving the structural profile unchanged where vocabulary dropout reshapes it.

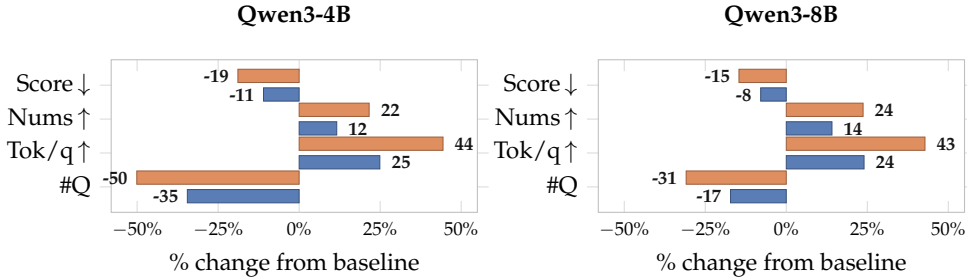


Figure 2: Question profile at iteration 5 (% change from baseline). ■ VD85 ■ VD75.

6.3 Curriculum quality in isolation

The solver gains in Table 1 arise inside the co-evolution loop, which couples curriculum content with self-consistency reward noise and the live proposer-solver pairing. To test whether the gains come from the curriculum itself, we isolate it. We take the iteration-5 question sets from the baseline, KL-Cov, Clip-Cov, and VD ($\alpha=0.75$) 8B runs, re-verify every problem with an external model (GPT-5-mini (Singh et al., 2025)), keep only those whose independently-derived answer matches the curriculum’s, roughly 20–31% of each arm, and GRPO-train a fresh Qwen3-4B-Base on each verified subset (training details in §D.6).

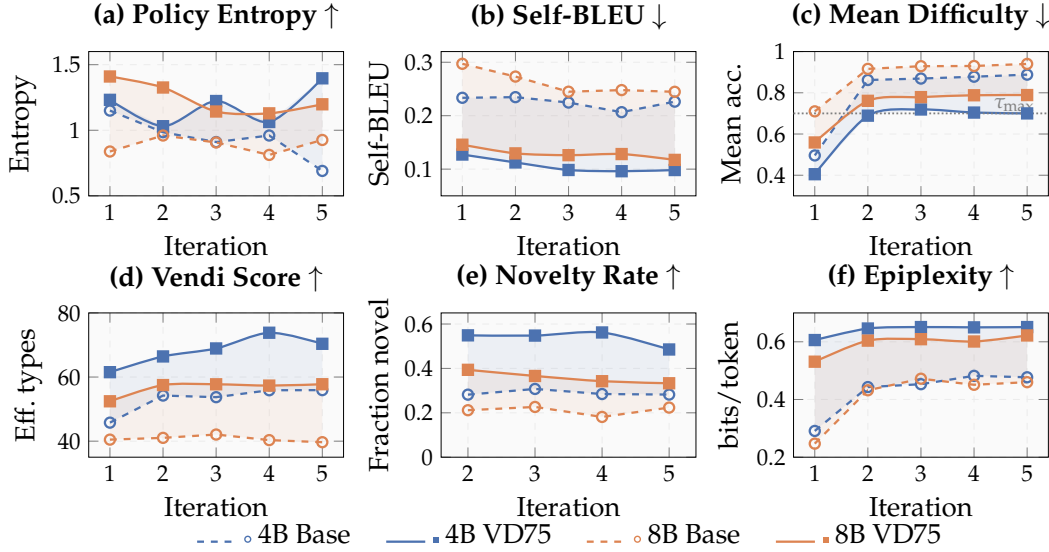


Figure 3: Diversity and curriculum quality over co-evolution iterations ($\alpha=0.75$, both phases). Top: collapse signals. Bottom: diversity metrics. Dashed = baseline, solid = dropout. Semantic metrics (b, d, e) use text-embedding-3-small (OpenAI, 2024).

Table 2: Solver pass@1 (%) after GRPO on each arm’s externally-verified iteration-5 curriculum, training a fresh Qwen3-4B-Base. Curricula are taken from the 8B co-evolution runs and verified with GPT-5-mini. Best trained arm per column in **green bold**.

Trained on	MATH500	GSM8K	AMC	Olympiad	AIME’24	AIME’25	Minerva	Avg.
Base (no training)	55.0	75.1	25.0	20.0	6.7	3.3	25.4	30.1
Baseline	71.0	88.6	40.0	28.2	3.3	6.7	29.8	38.2
KL-Cov	69.8	88.6	45.0	27.6	10.0	3.3	31.6	39.4
Clip-Cov	66.6	86.7	40.0	24.2	10.0	3.3	32.0	37.5
VD	71.0	89.8	50.0	27.8	3.3	10.0	37.1	41.3

All four trained arms improve substantially over the untrained base, so the comparison ranks curricula that each carry real signal. VD’s curriculum produces the strongest student of the four, beating the baseline curriculum by +3.1 on average and the next-best arm (KL-Cov) by +1.9. Clip-Cov’s curriculum falls below the baseline, so once verification noise is stripped its content is worse than R-Zero’s self-judged data. VD’s advantage holds even with externally verified answers and a fresh model, so it comes from the curriculum it produces rather than the self-consistency reward.

Diversity is not an artifact of ill-posedness. Generation-phase dropout lowers the proposer’s answer correctness by roughly 7% at convergence (Table 6), which raises the question of whether the added diversity simply reflects more ill-posed problems. To check, we isolate the well-posed portion of each curriculum at the final iteration, having GPT-4.1-mini label every problem in the trained difficulty band as well-posed or ill-posed and solve it independently (details in §C.4).

Table 3: Curriculum composition and diversity at the final iteration. *Filter pass* and *Pass rate* count generated problems landing in the difficulty band $[0.3, 0.7]$, *Well-posed* and *Correct* are GPT-4.1-mini judgments, and Δ Vendi is VD75 minus baseline on the well-posed (WP) and correct (C) subsets.

Scale	Method	Filter pass	Pass rate	Well-posed	Correct	Δ Vendi (WP)	Δ Vendi (C)
4B	Baseline	946	11.8%	850	275	—	—
	VD75	1,222	15.3%	925	269	+11.4	+7.1
8B	Baseline	510	6.4%	465	163	—	—
	VD75	1,408	17.6%	1,112	325	+7.5	+2.8

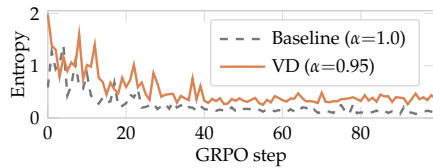
Vocabulary dropout produces more usable training data (Table 3). At 8B the baseline lands only 6.4% of its generated problems in the difficulty band, against VD75’s 17.6%, and VD75 yields roughly twice as many oracle-correct problems per iteration (325 vs. 163). The diversity gain also survives the correctness control. VD75’s Vendi exceeds the baseline by +11.4 at 4B and +7.5 at 8B on the well-posed subset, NLI logical diversity favors VD75 on the same subset (§C.4), and both gaps persist on the strictly-correct problems, so the added diversity reflects a more varied well-posed curriculum rather than extra noise.

6.4 Generality beyond math and co-evolution

Out-of-domain transfer. The results so far measure mathematical reasoning inside the R-Zero loop, so we also evaluate the 8B VD75 solver zero-shot on BBH (Suzgun et al., 2023), MedQA-USMLE (Jin et al., 2021), SuperGPQA (Team, 2025), and WinoGrande (Sakaguchi et al., 2020), none of which appear in training. The solver beats the R-Zero baseline on three of the four, by +10.1 on BBH, +5.0 on MedQA-USMLE, and +2.1 on SuperGPQA, trailing by 4.5 on WinoGrande (Table 4). Vocabulary dropout constrains the proposer’s output distribution rather than encoding anything math-specific, and the gains from its curriculum carry to domains the training set never covers.

Table 4: **Left:** zero-shot out-of-domain transfer of the 8B solver (VD at $\alpha=0.75$), evaluated under the protocol of Table 1. Best between Baseline and VD per row in **green bold**. **Right:** policy mean per-token entropy over the first 100 GRPO steps of single-policy training (Qwen3-4B-Base on GSM8K), baseline vs. VD ($\alpha=0.95$).

Benchmark	Base	Baseline	VD	Δ
BBH	37.5	44.8	55.0	+10.1
MedQA-USMLE	41.2	60.8	65.8	+5.0
SuperGPQA	27.0	27.7	29.8	+2.1
WinoGrande	66.1	69.1	64.6	-4.5



A single policy without self-play. To probe the mechanism outside co-evolution entirely, we train Qwen3-4B-Base on GSM8K with standard GRPO and a binary correctness reward, with and without a light mask ($\alpha=0.95$) on the policy’s rollouts (setup and per-benchmark results in §D.8). Accuracy stays close, with dropout ahead on four of seven benchmarks, and the clear effect is on entropy (Table 4). The unregularized policy collapses from 0.59 to 0.10, the failure mode documented by Cui et al. (2025), while vocabulary dropout holds entropy roughly $2\times$ higher on average and $3.8\times$ higher at convergence. Even without a co-evolving partner, the mask acts as an exploration regularizer, the same mechanism we argue sustains proposer diversity in the loop.

7 Discussion

Action-space constraints for language self-play. Our results support the hypothesis that explicit action-space constraints sustain productive co-evolution in language. R-Zero already

includes a reward-side repetition penalty based on pairwise BLEU, and the baseline proposer still collapses to a narrow distribution within two iterations, so reward-side diversity pressure alone appears insufficient. Vocabulary dropout constrains the output space directly instead, and because the mask moves every batch there is no fixed template distribution for the proposer to settle into. The diversity gains (§6.2), the solver improvements (Table 1), and the slower entropy collapse outside co-evolution (§6.4) are all consistent with this explanation.

Curriculum quality vs. curriculum diversity. Proposer correctness degrades in every condition as training progresses, from 56–81% at the first iteration to roughly 32–39% by the last (Table 6). The degradation reflects a limit of self-consistency verification that the baseline and vocabulary dropout share, namely that solver disagreement cannot separate hard problems from ill-posed ones. Diversity collapse is the orthogonal problem, and that is the one vocabulary dropout addresses. The baseline produces a narrow, repetitive curriculum of declining quality, while dropout maintains a varied one whose well-posed subset is rich enough to offset the noise. Shao et al. (2025) report that GRPO can amplify pre-training priors even under spurious rewards, and Setlur et al. (2024) find that incorrect synthetic data can still help when negative signals are well structured, both consistent with what we observe. Stronger verification layered on top could keep the diversity while filtering the noise (§C.2).

Non-stationarity and scale. The masking-strength ablations (Table 1) show that vocabulary dropout’s value is scale-dependent, a capacity effect. Larger models carry more redundant token representations and can reroute around masked tokens, so a more aggressive mask raises diversity without sacrificing coherence. This points toward adaptive masks that scale α with model size or anneal it over training. A linear anneal from $\alpha=0.75$ to 1.0 supports this reading, peaking two iterations earlier than fixed VD75, winning three of five iterations on math average, and tracing a more stable trajectory, with an OlympiadBench range of 3.0 points against 6.4 for fixed masking (§D.7). It also attains the highest cumulative performance across the five iterations.

Scope and limitations. When the proposer already has more capacity than the solver (8B→4B cross-scale, §D.3), dropout hurts rather than helps (−1.4 avg), because the difficulty calibration is already misaligned and diversity pressure amplifies the mismatch. Even without dropout, the cross-scale pairing does no better than the symmetric baseline (38.8 vs. 38.3 avg), so a stronger proposer alone does not buy a better curriculum. The matched-scale 8B curriculum still trains a 4B model well in static transfer (§6.3), so the live difficulty calibration rather than the curriculum content drives this degradation. Similarly, co-evolution with or without dropout does not consistently benefit instruction-tuned models in our Qwen2.5-1.5B-Instruct experiments (§D.4). These boundaries track the co-evolutionary setting rather than vocabulary dropout itself. R-Zero-style self-play is most productive when proposer and solver are capacity-matched base models, the range in which vocabulary dropout operates.

8 Conclusion

We showed that a simple, non-stationary constraint on the proposer’s output vocabulary sustains curriculum diversity and improves solver performance in co-evolutionary self-play. Like dropout on hidden units, which prevents co-adaptation of neurons, it softens co-adaptation of token sequences. The method needs no auxiliary models or reward modifications, keeps the curriculum from collapsing onto trivially easy problems, and its gains carry beyond math at 8B (§6.4). Our results suggest that action-space structure, rather than reward design alone, is a productive lever for co-evolutionary dynamics in language, and composing the mask with stronger verification is a natural route to self-play curricula that stay both varied and reliable.

Ethics Statement

This work studies training dynamics in co-evolutionary LLM systems using mathematical reasoning as a testbed. The models, data, and methods are standard in the field. We do not foresee specific negative societal consequences beyond those common to language model research.

Reproducibility Statement

All of our code is released at https://github.com/ARC-ASU/vocab_dropout, with runbooks covering the vocabulary dropout runs, the R-Zero baseline, and the KL-Cov and Clip-Cov arms. The same repository holds the diversity metric implementations and pins the dependency versions we ran with. Hyperparameters, prompt templates, and the verification procedure appear in §E, §F, and §B.1. Reported numbers are means over 3 seeds with standard errors.

References

- Luke Bailey, Kaiyue Wen, Kefan Dong, Tatsunori Hashimoto, and Tengyu Ma. Scaling self-play with self-guidance, 2026. URL <https://arxiv.org/abs/2604.20209>.
- Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.
- Gavin C. Cawley and Nicola L.C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *J. Mach. Learn. Res.*, 11:2079–2107, August 2010. ISSN 1532-4435.
- Justin Yang Chae, Md Tanvirul Alam, and Nidhi Rastogi. Towards understanding self-play for llm reasoning. *arXiv preprint arXiv:2510.27072*, 2025.
- Yash Chandak, Georgios Theodorou, James Kostas, Scott Jordan, and Philip Thomas. Learning action representations for reinforcement learning. In *International conference on machine learning*, pp. 941–950. PMLR, 2019.
- Yixing Chen, Yiding Wang, Siqi Zhu, Hao-fei Yu, Tao Feng, Muhan Zhang, Mostofa Patwary, and Jiaxuan You. Multi-agent evolve: Llm self-improve through co-evolution. *arXiv preprint arXiv:2510.23595*, 2025.
- Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 6621–6642. PMLR, 2024. URL <https://proceedings.mlr.press/v235/chen24j.html>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Jacob Dineen, Aswin RRV, Qin Liu, Zhikun Xu, Xiao Ye, Ming Shen, Zhaonan Li, Shijie Lu, Chitta Baral, Muhao Chen, and Ben Zhou. QA-LIGN: Aligning LLMs through constitutionally decomposed QA. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 20619–20642. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.1123. URL <https://aclanthology.org/2025.findings-emnlp.1123/>.

- Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A Smith. Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 2185–2194, 2019.
- Gregory Farquhar, Laura Gustafson, Zeming Lin, Shimon Whiteson, Nicolas Usunier, and Gabriel Synnaeve. Growing action spaces. In *International Conference on Machine Learning*, pp. 3040–3051. PMLR, 2020.
- Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J Zico Kolter, and Andrew Gordon Wilson. From entropy to epiplexity: Rethinking information for computationally bounded intelligence. *arXiv preprint arXiv:2601.03220*, 2026.
- Dan Friedman and Adji Bousso Dieng. The vendi score: A diversity evaluation metric for machine learning. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=g970HbQyk1>.
- Jingchu Gai, Guanning Zeng, Huaqing Zhang, and Aditi Raghunathan. Differential smoothing mitigates sharpening and improves llm reasoning. *arXiv preprint arXiv:2511.19942*, 2025.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3828–3850, 2024.
- Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games. *arXiv preprint arXiv:1603.01121*, 2016.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/be83ab3ecd0db773eb2dc1b0a17836a1-Paper-round2.pdf.
- Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-zero: Self-evolving reasoning LLM from zero data. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=96apU6YzSO>.
- Hugging Face. Math-verify: Robust mathematical expression evaluator. <https://github.com/huggingface/Math-Verify>, 2025. GitHub repository; accessed 2025-09-09.
- Prime Intellect. Prime-rl, 2025. URL <https://github.com/PrimeIntellect-ai/prime-rl>.
- Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, et al. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.
- Liwei Jiang, Yuanjun Chai, Margaret Li, Mickel Liu, Raymond Fok, Nouha Dziri, Yulia Tsvetkov, Maarten Sap, Alon Albalak, and Yejin Choi. Artificial hivemind: The open-ended homogeneity of language models (and beyond). *arXiv preprint arXiv:2510.22954*, 2025.
- Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021. doi: 10.3390/app11146421. URL <https://www.mdpi.com/2076-3417/11/14/6421>.
- Constantinos Karouzos, Xingwei Tan, and Nikolaos Aletras. Where does output diversity collapse in post-training? *arXiv preprint arXiv:2604.16027*, 2026.

- Haziq Mohammad Khalid, Salsabeel Shapsough, and Imran Zualkernan. Noise steering for controlled text generation: Improving diversity and reading-level fidelity in arabic educational story generation. *arXiv preprint arXiv:2604.03380*, 2026.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in neural information processing systems*, 35:3843–3857, 2022.
- Tianjian Li, Yiming Zhang, Ping Yu, Swarnadeep Saha, Daniel Khashabi, Jason Weston, Jack Lanchantin, and Tianlu Wang. Jointly reinforcing diversity and quality in language model generations. *arXiv preprint arXiv:2509.02534*, 2025.
- Bo Liu, Chuanyang Jin, Seungone Kim, Weizhe Yuan, Wenting Zhao, Ilia Kulikov, Xian Li, Sainbayar Sukhbaatar, Jack Lanchantin, and Jason Weston. Spice: Self-play in corpus environments improves reasoning. *arXiv preprint arXiv:2510.24684*, 2025.
- Wei Liu, Siya Qi, Yali Du, and Yulan He. Self-play only evolves when self-synthetic pipeline ensures learnable information gain. *arXiv preprint arXiv:2603.02218*, 2026.
- Xueguang Ma, Qian Liu, Dongfu Jiang, Ge Zhang, Zejun Ma, and Wenhui Chen. General-reasoner: Advancing llm reasoning across all domains. *arXiv preprint arXiv:2505.14652*, 2025.
- Mathematical Association of America. American mathematics competitions (AMC) 2023. <https://artofproblemsolving.com/wiki/index.php/AMC>, 2023.
- Mathematical Association of America. American invitational mathematics examination (AIME) 2024. https://artofproblemsolving.com/wiki/index.php/2024_AIME_I, 2024.
- Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.
- Vaibhav Mishra. Preventing curriculum collapse in self-evolving reasoning systems. *arXiv preprint arXiv:2603.13309*, 2026.
- Yuta Nozaki, Dai Nakashima, Ryo Sato, Naoki Asaba, and Shintaro Kawamura. Efficient vocabulary reduction for small language models. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, Steven Schockaert, Kareem Darwish, and Apoorv Agarwal (eds.), *Proceedings of the 31st International Conference on Computational Linguistics: Industry Track*, pp. 771–783, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.coling-industry.64/>.
- OpenAI. New embedding models and API updates. <https://openai.com/index/new-embedding-models-and-api-updates/>, 2024. Accessed: 2025-06-01.
- OpenAI. GPT-4.1 and GPT-4.1 mini. <https://openai.com/index/gpt-4-1/>, 2025. Accessed: 2025-06-01.
- Jing-Cheng Pang, Liang Lu, Xian Tang, Kun Jiang, Sijie Wu, Kai Zhang, and Xubin Li. Reinforcement learning with promising tokens for large language models. *arXiv preprint arXiv:2602.03195*, 2026.
- Sophia Xiao Pu, Zhaotian Weng, Chengzhi Liu, Jayanth Srinivasa, Gaowen Liu, William Yang Wang, and Xin Eric Wang. Survive or collapse: The asymmetric roles of data gating and reward grounding in self-play rl, 2026. URL <https://arxiv.org/abs/2605.22217>.

- Aswin RRV, Jacob Dineen, Divij Handa, Md Nayem Uddin, Mihir Parmar, Chitta Baral, and Ben Zhou. Thinktuning: Instilling cognitive reflections without distillation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 31236–31250, 2025.
- Aswin RRV, Jacob Dineen, Divij Handa, Mihir Parmar, Ben Zhou, Swaroop Mishra, and Chitta Baral. Mid-training with self-generated data improves reinforcement learning in language models, 2026. URL <https://arxiv.org/abs/2605.08472>.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pp. 8732–8740. AAAI Press, 2020. doi: 10.1609/AAAI.V34I05.6399. URL <https://doi.org/10.1609/aaai.v34i05.6399>.
- Amrith Setlur, Saurabh Garg, Xinyang Geng, Naman Garg, Virginia Smith, and Aviral Kumar. RL on incorrect synthetic data scales the efficiency of llm math reasoning by eight-fold. *Advances in Neural Information Processing Systems*, 37:43000–43031, 2024.
- Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, et al. Spurious rewards: Rethinking training signals in rlvr. *arXiv preprint arXiv:2506.10947*, 2025.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shashank Sharma, Janina Hoffmann, and Vinay Namboodiri. Ollm: Options-based large language models, 2026. URL <https://arxiv.org/abs/2604.19087>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354–359, October 2017. URL <http://dx.doi.org/10.1038/nature24270>.
- Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- Katherine Stasaski and Marti Hearst. Semantic diversity in dialogue with natural language inference. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 85–98, Seattle, United States, July 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.naacl-main.6. URL <https://aclanthology.org/2022.naacl-main.6/>.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. In Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 9-14, 2023*, Findings of ACL, pp. 13003–13051. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.FINDINGS-ACL.824. URL <https://doi.org/10.18653/v1/2023.findings-acl.824>.
- M-A-P Team. Supergpqa: Scaling LLM evaluation across 285 graduate disciplines. *CoRR*, abs/2502.14739, 2025. doi: 10.48550/ARXIV.2502.14739. URL <https://doi.org/10.48550/arXiv.2502.14739>.

- Gerald Tesauro. Temporal difference learning and td-gammon. *Commun. ACM*, 38(3):58–68, March 1995. ISSN 0001-0782. doi: 10.1145/203330.203343. URL <https://doi.org/10.1145/203330.203343>.
- Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Joseph Dudzik, Junyoung Chung, David Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, L. Sifre, Trevor Cai, John P. Agapiou, Max Jaderberg, Alexander Sasha Vezhnevets, Rémi Leblond, Tobias Pohlen, Valentin Dalibard, David Budden, Yury Sulsky, James Molloy, Tom Le Paine, Caglar Gulcehre, Ziyun Wang, Tobias Pfaff, Yuhuai Wu, Roman Ring, Dani Yogatama, Dario Wünsch, Katrina McKinney, Oliver Smith, Tom Schaul, Timothy P. Lillicrap, Koray Kavukcuoglu, Demis Hassabis, Chris Apps, and David Silver. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575:350 – 354, 2019. URL <https://api.semanticscholar.org/CorpusID:204972004>.
- Shaobo Wang, Zhengbo Jiao, Zifan Zhang, Yilang Peng, Xu Ze, Boyu Yang, Wei Wang, Hu Wei, and Linfeng Zhang. Socratic-zero: Bootstrapping reasoning via data-free agent co-evolution. *arXiv preprint arXiv:2509.24726*, 2025a.
- Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b. URL <https://openreview.net/forum?id=yfcpdY4gMP>.
- Alex Wilf, Pranjal Aggarwal, Bryan Parno, Daniel Fried, Louis-Philippe Morency, Paul Pu Liang, and Sean Welleck. Propose, solve, verify: Self-play through formal verification. *arXiv preprint arXiv:2512.18160*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Xiao Ye, Shaswat Shrivastava, Zhaonan Li, Jacob Dineen, Shijie Lu, Avneet Ahuja, Ming Shen, Zhikun Xu, and Ben Zhou. Cc-learn: Cohort-based consistency learning, 2025. URL <https://arxiv.org/abs/2506.15662>.
- Wenhao Yu, Zhenwen Liang, Chengsong Huang, Kishan Panaganti, Tianqing Fang, Haitao Mi, and Dong Yu. Guided self-evolving llms with minimal human supervision. *arXiv preprint arXiv:2512.02472*, 2025.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=40sgYD7em5>.
- Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=neZSGqhxDa>.
- Yang Zhou, Sunzhu Li, Shunyu Liu, Wenkai Fang, Kongcheng Zhang, Jiale Zhao, Jingwen Yang, Yihe Zhou, Jianwei Lv, Tongya Zheng, et al. Breaking the exploration bottleneck: Rubric-scaffolded reinforcement learning for general llm reasoning. *arXiv preprint arXiv:2508.16949*, 2025.

Appendix Contents

A Algorithm	17
B Implementation details	17
B.1 Verification via solver self-consistency	17
B.2 Vocabulary dropout implementation	18
B.3 Compute requirements	18
B.4 Entropy-control baselines	18
C Qualitative examples	19
C.1 Baseline vs. vocabulary dropout	19
C.2 Ill-posed problems	19
C.3 Proposer answer correctness	20
C.4 Curriculum diversity controlling for correctness	21
D Additional results	21
D.1 Cumulative embedding diversity	21
D.2 Phase ablation embedding analysis	22
D.3 Cross-scale co-evolution	22
D.4 Instruction-tuned models	24
D.5 Vocabulary utilization	24
D.6 Curriculum-transfer training details	24
D.7 Annealed masking schedule	24
D.8 Vocabulary dropout outside co-evolution	25
E Hyperparameters	26
E.1 Evaluation benchmarks	26
E.2 Training hyperparameters	26
E.3 Vendi score computation	26
E.4 Epiplexity computation	27
F Prompt templates	27
F.1 Original R-Zero questioner prompt	27
F.2 Evaluation prompt	28

A Algorithm

Algorithm 1 gives the full co-evolutionary training loop.

Algorithm 1: Vocabulary Dropout Co-Evolutionary Self-Play

Input : Base model θ_0 ;
retention prob. α ;
difficulty window $[\tau_{\min}, \tau_{\max}]$;
iterations T ;
proposer GRPO group size G ;
solver self-consistency samples M

Output: Trained proposer $\pi_P^{(T)}$ and solver $\pi_S^{(T)}$

```

1  $\pi_P^{(0)}, \pi_S^{(0)} \leftarrow \theta_0$ 
2  $\mathcal{F} \leftarrow$  protected format-critical tokens // always survive
3 for  $t = 1, \dots, T$  do
4   /* Phase 1: Train proposer (solver frozen) */
5   for each GRPO batch do
6      $\mathcal{M}^{(b)} \leftarrow \{v \in \mathcal{V} : m_v \sim \text{Bern}(\alpha)\} \cup \mathcal{F}$  // fresh mask per batch
7     Sample  $\{o_i\}_{i=1}^G$  from  $\pi_P$ , setting logits to  $-\infty$  for tokens  $\notin \mathcal{M}^{(b)}$ 
8     for each rollout  $o_i$  do
9       Parse  $q_i, a_i$  from  $o_i$  // question, boxed answer
10      if parse fails then  $r_i^P \leftarrow 0$ ;
11      continue
12      Query frozen  $\pi_S^{(t-1)}$  with  $q_i$ ;
13      collect  $M$  responses
14       $\text{acc}_i \leftarrow$  fraction matching majority vote
15       $r_i^P \leftarrow \min(\text{acc}_i, 1 - \text{acc}_i) \cdot \mathbf{1}[\text{acc}_i \in [\tau_{\min}, \tau_{\max}]]$ 
16    end
17    Update  $\pi_P$  via GRPO on  $\{(o_i, r_i^P)\}$ 
18   $\pi_P^{(t)} \leftarrow \pi_P$ 
19  /* Phase 2: Generate curriculum, then train solver */
20   $\mathcal{Q} \leftarrow$  sample problems from  $\pi_P^{(t)}$  with per-batch  $\mathcal{M}^{(b)}$ 
21   $\mathcal{Q}_{\text{train}} \leftarrow \{(q, a) \in \mathcal{Q} : \text{acc}(q) \in [\tau_{\min}, \tau_{\max}]\}$  //  $a$  is the proposer's stated answer
22  for each GRPO batch do
23    Sample solver rollouts for  $(q, a) \in \mathcal{Q}_{\text{train}}$ 
24     $r^S \leftarrow \mathbf{1}[\text{solver answer} = a]$ 
25    Update  $\pi_S$  via GRPO on  $\{(q, r^S)\}$ 
26  end
27   $\pi_S^{(t)} \leftarrow \pi_S$ 
28 end
29 return  $\pi_P^{(T)}, \pi_S^{(T)}$ 

```

B Implementation details

B.1 Verification via solver self-consistency

Following R-Zero (Huang et al., 2026), we verify proposer outputs using the solver itself rather than an external reward model. Each proposer output is checked for valid format

(<question> tags and a \boxed{} answer). Valid questions are sent to the frozen solver, which attempts each problem 10 times. The proposer reward is $r = \min(\text{acc}, 1 - \text{acc})$, where acc is the fraction of solver responses matching the majority vote. This peaks at 50% solve rate, incentivizing problems at the solver’s decision boundary.

For solver training, generated questions are filtered to the difficulty window $[\tau_{\min}, \tau_{\max}] = [0.3, 0.7]$, following R-Zero.

B.2 Vocabulary dropout implementation

Vocabulary dropout rides on vLLM’s `allowed_token_ids` parameter in `SamplingParams`, which sets logits to $-\infty$ for every token outside the allowed set before sampling. At the start of each batch we draw a fresh Bernoulli mask over the full vocabulary, add the protected set $\mathcal{F}$ of format-critical tokens needed for structural validity and answer formatting, and hand the resulting token ID list to vLLM. Nothing in the model weights or the sampling kernel changes. Figure 4 shows the code change relative to the standard R-Zero rollout.

```
def rollout(model, prompts,
-   sampling_params):
+   sampling_params,
+   tokenizer, alpha=0.75):
+   V = tokenizer.vocab_size
+   mask = torch.bernoulli(
+       torch.full((V,), alpha)
+   ).bool()
+   # Protect format-critical tokens
+   for t in get_protected_token_ids(
+       tokenizer):
+       mask[t] = True
+   sampling_params.allowed_token_ids = (
+       mask.nonzero(as_tuple=True)[0]
+       .tolist())
    completions = model.generate(
        prompts, sampling_params
    )
    return completions
```

Figure 4: Vocabulary dropout as a unified diff. The only change is sampling a Bernoulli mask and passing the surviving token IDs to vLLM’s `allowed_token_ids` before generation. The mask is resampled every batch.

B.3 Compute requirements

Every run uses 2 NVIDIA H200 GPUs. In the proposer phase one GPU handles GRPO training and the other hosts a vLLM server for self-consistency scoring, while curriculum generation and question evaluation split across both. The solver phase trains on both GPUs. Each experiment covers 5 iterations and 3 independent seeds per configuration, and benchmark evaluation runs on 4–8 NVIDIA H100 GPUs through vLLM.

Vocabulary dropout itself adds no compute beyond the standard rollout, since the mask is enforced at the logit level inside the existing sampling kernel.

B.4 Entropy-control baselines

We compare against the two entropy-control methods of Cui et al. (2025), which prevent policy entropy collapse by limiting updates on tokens whose log-probability and advantage co-vary most strongly. KL-Cov adds a per-token KL penalty to the top- k such tokens

(their Eq. 14), while Clip-Cov detaches the gradient on a random subset of tokens whose covariance falls in $[\omega_{\text{low}}, \omega_{\text{high}}]$, leaving the standard PPO loss elsewhere. Both are applied to the proposer only, matching VD’s scope, with all other training settings identical to the Baseline and VD arms. We list the hyperparameters in Table 5.

We also tried the simplest diversity intervention, raising the proposer’s sampling temperature. At $T \geq 1.5$ the proposer degrades enough that it cannot populate a full batch of valid question-answer pairs, so R-Zero’s filtering and extraction break in the first co-evolution iteration. We therefore do not report higher temperature as a baseline.

Table 5: Entropy-control hyperparameters, following Cui et al. (2025)’s 7B reference configuration.

Method	Hyperparameter	Value
KL-Cov	k (top-covariance token fraction)	2×10^{-3}
	β (KL coefficient on selected tokens)	1.0
Clip-Cov	ρ (masked-token fraction)	2×10^{-4}
	$\omega_{\text{low}}, \omega_{\text{high}}$ (covariance bounds)	1.0, 5.0
Both	Entropy bonus coefficient	0
	Scope	Proposer only

C Qualitative examples

We present representative problems generated by the baseline and vocabulary dropout proposers at the same co-evolution iteration, as well as examples of ill-posed problems that both proposers produce.

C.1 Baseline vs. vocabulary dropout

The following pairs are drawn from iteration-5 question sets of the 8B proposer. For each pair, the baseline and dropout examples share the same broad semantic theme. Dropout examples were filtered to those with solver accuracy ≥ 0.7 .

Baseline 8B – Baking

Sarah is baking cookies for her class. She has 5 trays, and each tray can hold 12 cookies. If she wants to bake 60 cookies in total, how many batches does she need to bake if each batch uses all 5 trays?

VD75 8B – Baking

Sarah is baking cookies to sell at the bake sale to raise funds for the kindergarten program at her local elementary school. She plans to bake three types: sugar cookies, oat cakes, and granola bars. She baked 120 cookies overall. If half are sugar cookies and four-fifths of the remaining are oat cakes, how many granola bars did she bake?

Baseline 8B – Farming

A farmer has a rectangular field that is 30 meters long and 20 meters wide. He wants to plant apple trees in rows spaced 5 meters apart and plants 3 trees in each row. How many apple trees can the farmer plant in his field?

VD75 8B – Farming

At the local farmer stand, apples are sold at \$2 per kilo and bananas at \$3 per kilo. A customer bought 1 kilo of apples and 1 kilo of bananas on Monday, and 2 kilos of apples and 2 kilos of bananas on Tuesday. What was the customer’s total expenditure over the two days?

C.2 Ill-posed problems

Self-consistency verification rewards solver disagreement, but cannot distinguish hard problems from ill-defined ones. Both the baseline and vocabulary dropout proposers produce problems where the proposer’s stated ground truth is incorrect. In each case below, all 10 solvers answered correctly but were scored as wrong (score = 1.0) because they disagreed with the proposer’s erroneous answer.

Baseline 8B <i>Sophie bought 126 seeds. She planted 34 on Saturday and 58 on Sunday. How many left? Prop: 92 Solvers: 34 ✓ (10/10)</i> <i>Subtracted only Saturday; forgot Sunday. Correct: $126 - 34 - 58 = 34$.</i>	VD75 8B <i>Alice has 48 marigolds and 36 roses, bundles them equally (largest possible). How many marigolds per bundle? Prop: 48 Solvers: 12 ✓ (10/10)</i> <i>Wrote total count (48) instead of bundle size (GCD = 12).</i>
Baseline 8B <i>It takes 30 seconds to fill a 6-liter jug. How many liters in 2 minutes? Prop: 4 Solvers: 24 ✓ (10/10)</i> <i>Computed fills ($120 \div 30 = 4$) but forgot jug capacity. Correct: $4 \times 6 = 24$.</i>	VD75 8B <i>Mike paints a fence in 2 hr, Tom in 3 hr. How many minutes together? Prop: 1 min... Solvers: 72 ✓ (10/10)</i> <i>Confused hourly with per-minute rates; answer truncated. Correct: 72 min.</i>
Baseline 4B <i>John had 50 dollars. He bought a 10 dollar ice cream. How many dollars left? Prop: 50 Solvers: 40 ✓ (10/10)</i> <i>Echoed starting amount without subtracting. Correct: $50 - 10 = 40$.</i>	VD75 4B <i>Simon had 30 cars at 12, half of age-15 count. Simon had half of Lou's at 15. Lou's total? Prop: 60 Solvers: 120 ✓ (10/10)</i> <i>Stopped one step early; attributed Simon's count to Lou. Correct: $60 \times 2 = 120$.</i>
Baseline 4B <i>180 pages in a book. Emma reads $\frac{1}{5}$ in week 1 and $\frac{2}{5}$ in week 2. How many in week 2? Prop: 36 Solvers: 72 ✓ (10/10)</i> <i>Computed week-1 fraction instead of week-2. Correct: $\frac{2}{5} \times 180 = 72$.</i>	VD75 4B <i>John bought 4 fish, returned half. How many left? Prop: 4 Solvers: 2 ✓ (10/10)</i> <i>Echoed starting count without applying return. Correct: $4 - 2 = 2$.</i>

These failures arise from a limitation of self-consistency verification, which equates solver disagreement with problem difficulty regardless of answer correctness. The proposer receives maximum reward for these problems precisely because it provided a wrong answer that no solver reproduces. Both the baseline and dropout settings exhibit this failure mode, which is a property of the verification scheme rather than vocabulary dropout.

C.3 Proposer answer correctness

To estimate the fraction of generated problems where the proposer's stated answer is correct, we use GPT-4.1-mini (OpenAI, 2025) as an independent verifier. For each problem at every iteration, the verifier solves the problem independently and compares its answer to the proposer's. Table 6 reports correctness trajectories across all conditions and phase ablations.

Table 6: Proposer answer correctness (%) over co-evolution iterations, judged by GPT-4.1-mini solving each problem independently.

Model	Setting	It. 1	It. 2	It. 3	It. 4	It. 5
Qwen3-8B	Baseline	81.4	45.5	42.2	39.1	38.0
	Train-only	79.0	43.2	42.0	40.4	39.0
	Gen-only	62.0	38.3	33.7	33.4	31.9
	VD75	61.8	34.9	33.1	36.7	32.4
Qwen3-4B	Baseline	75.6	38.1	38.0	42.2	37.7
	Train-only	76.0	37.2	44.2	38.0	38.2
	Gen-only	56.0	34.5	32.1	29.5	32.4
	VD75	61.2	31.2	30.5	30.7	31.9

Table 7: Proposer reasoning ability (pass@1 %) on standard benchmarks. VD is applied to the proposer during co-evolution. Co-evolutionary training does not improve the proposer’s own problem-solving ability.

Setting	Pass@1 (%)						Avg.
	MATH500	GSM8K	AMC	Olympiad	AIME'24	AIME'25	
Qwen3-4B							
Base (no training)	54.1 ± 2.2	63.5 ± 5.2	29.2 ± 1.8	20.6 ± 1.3	12.2 ± 0.9	1.1 ± 0.9	30.1
Baseline ($\alpha=1.0$)	56.0 ± 1.1	72.7 ± 3.4	33.3 ± 3.0	20.3 ± 0.9	10.0 ± 1.6	4.4 ± 0.9	32.8
VD85 ($\alpha=0.85$)	58.4 ± 0.8	60.5 ± 4.1	37.5 ± 4.1	19.3 ± 1.1	11.1 ± 0.9	4.4 ± 2.4	31.9
Qwen3-8B							
Base (no training)	70.7 ± 0.1	82.2 ± 0.7	45.8 ± 3.6	18.1 ± 1.3	8.9 ± 2.4	7.8 ± 0.9	38.9
Baseline ($\alpha=1.0$)	65.1 ± 2.1	87.5 ± 1.2	40.8 ± 3.8	21.8 ± 0.7	7.8 ± 0.9	10.0 ± 1.6	38.8
VD75 ($\alpha=0.75$)	61.7 ± 1.7	85.2 ± 1.4	40.0 ± 3.1	23.3 ± 1.6	11.1 ± 0.9	8.9 ± 1.8	38.4

All conditions show a sharp correctness drop from iteration 1 to 2, then stabilize. This mirrors the stagnation pattern observed in §6.2, where the proposer’s distribution shifts rapidly in the first iteration as it learns to target the solver’s decision boundary, then locks in. Two clusters emerge at convergence. Conditions without generation-phase dropout (baseline, train-only) stabilize at $\sim 38\text{--}42\%$ correctness, while conditions with generation-phase dropout (gen-only, VD75) settle at $\sim 30\text{--}34\%$. The $\sim 7\%$ gap is the price of diversity, since the non-stationary mask produces more varied but less reliably correct problems. Despite this, VD75 produces the strongest solver improvements at 8B (Table 1), suggesting that the diversity of the well-posed subset more than compensates for the increased noise. This motivates composing vocabulary dropout with stronger verification, as discussed in §7.

C.4 Curriculum diversity controlling for correctness

This section gives the methodology behind Table 3 in §6.3. At the final co-evolution iteration we record how many generated problems land in the productive difficulty band $[\tau_{\min}, \tau_{\max}] = [0.3, 0.7]$ that the solver trains on, then label each filtered problem as well-posed (its answer is determinate, whether the proposer states it correctly or not) or ill-posed, using the same GPT-4.1-mini verifier as §C.3. Beyond the Vendi gaps in Table 3, NLI logical diversity on the well-posed subset also favors VD75, by +0.12 at 4B and +0.03 at 8B.

D Additional results

D.1 Cumulative embedding diversity

Figure 3 reports per-iteration Vendi scores and novelty rates. As a complementary view, Figure 5 tracks the cumulative Vendi score, where all questions from iterations 1 through N are pooled (subsampled to 2,000), embedded with text-embedding-3-small (OpenAI, 2024), and scored. VD75 maintains $\sim 35\%$ higher cumulative diversity than the baseline at both scales. The baseline plateaus after iteration 2–3, while VD75 continues to grow.

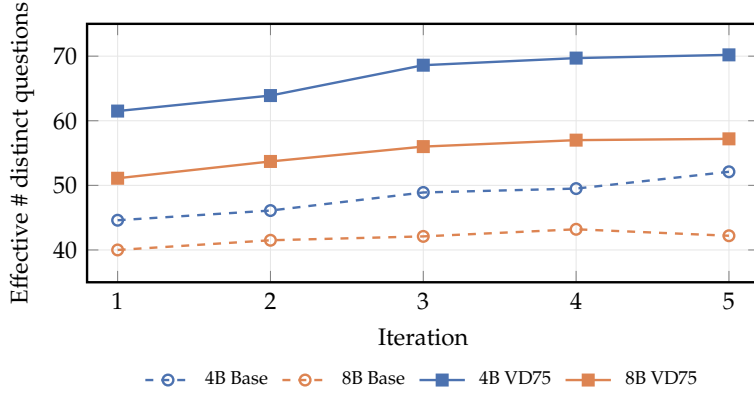


Figure 5: Cumulative Vendi score (questions pooled across iterations 1–N, subsampled to 2,000). VD75 maintains higher diversity at both scales.

As additional semantic-diversity views, Table 8 reports mean pairwise cosine distance with Qwen3-Embedding-4B and NLI-based logical diversity computed with DeBERTa-v3-large fine-tuned on MNLI/FEVER/ANLI/LingNLI/WANLI, on the curriculum-filtered problem sets at iteration 5. VD outperforms the baseline at every (scale, metric) cell, with relative lifts ranging from +4–6% (semantic) to +19–68% (logical). This rank order holds under an encoder swap (all-MiniLM-L6-v2) and unidirectional NLI scoring.

Table 8: NLI and cosine diversity at iteration 5, on the curriculum-filtered problem sets the solver trains on. Higher is better for both. Standard errors bootstrapped from 100 resamples. Best per scale in **green bold**.

Model	Setting	Semantic ↑		Logical ↑	
		Value	Δ	Value	Δ
Qwen3-4B	Baseline	$0.665 \pm <0.001$	—	$+0.241 \pm 0.013$	—
	VD	$0.692 \pm <0.001$	+4.1%	$+0.405 \pm 0.014$	+68.1%
Qwen3-8B	Baseline	$0.638 \pm <0.001$	—	$+0.315 \pm 0.013$	—
	VD	$0.677 \pm <0.001$	+6.1%	$+0.376 \pm 0.016$	+19.4%

D.2 Phase ablation embedding analysis

Table 9 tracks cumulative Vendi scores as questions are pooled across iterations, and Figure 6 shows per-iteration trends for all three embedding metrics. Gen-only dropout starts with high diversity but plateaus quickly (+4.6/+2.6 growth at 4B/8B), suggesting that, without a regularized solver, the proposer’s distribution converges. Train-only starts lower but sustains steady growth (+9.6/+4.2), indicating that the regularized solver feeds back different reward signals that prevent proposer collapse. Both phases combined yield the highest cumulative diversity and the strongest growth (+8.7/+6.1), consistent with the solver accuracy results in Table 1.

D.3 Cross-scale co-evolution

All experiments in §6 pair each model with itself (4B→4B, 8B→8B). A natural question is whether a stronger proposer generates a better curriculum for a weaker solver. We test this by pairing a Qwen3-8B proposer with a Qwen3-4B solver, with and without vocabulary dropout ($\alpha=0.85$, gen-only) on the 8B proposer.

Table 9: Cumulative question diversity (Vendi score) over co-evolution iterations via text-embedding-3-small embeddings. *Growth* is the gain from iteration 1 to iterations 1–5 pooled.

Model	Setting	Cumulative Vendi Score $\uparrow$					Growth
		It. 1	1–2	1–3	1–4	1–5	
Qwen3-4B	Baseline	44.6	46.1	48.9	49.5	52.1	+7.5
	Train-only	46.1	51.0	52.0	53.7	55.7	+9.6
	Gen-only	59.7	62.9	63.9	63.9	64.3	+4.6
	Both	61.5	63.9	68.6	69.7	70.2	+8.7
Qwen3-8B	Baseline	40.0	41.5	42.1	43.2	42.2	+2.2
	Train-only	41.9	42.6	43.5	44.2	46.1	+4.2
	Gen-only	52.9	54.1	55.2	55.3	55.5	+2.6
	Both	51.1	53.7	56.0	57.0	57.2	+6.1

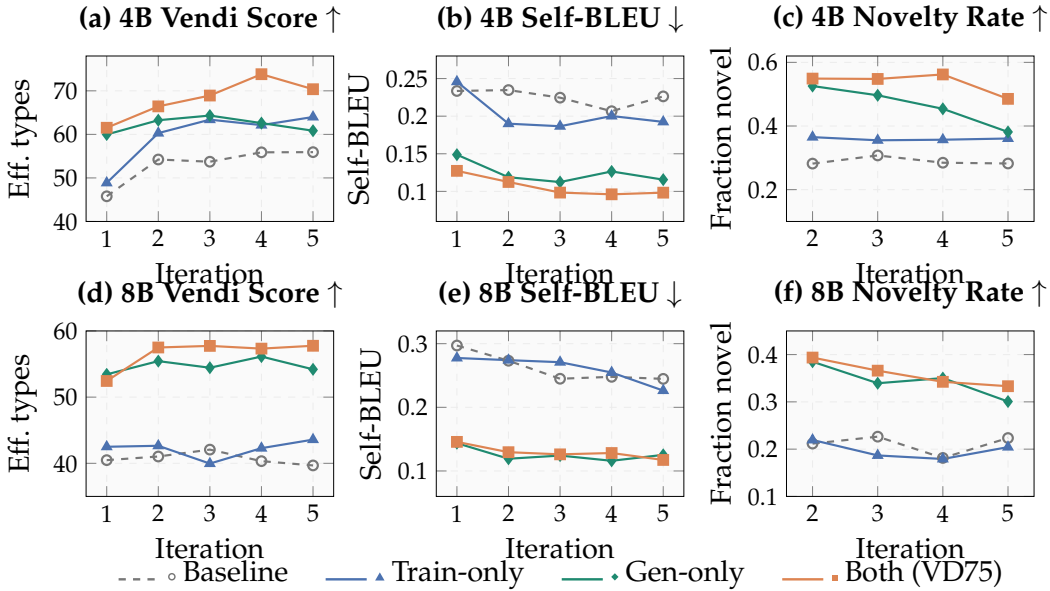


Figure 6: Embedding diversity by dropout phase ($\alpha=0.75$). Both phases combined achieves the highest diversity. All metrics use text-embedding-3-small.

Table 10: Cross-scale co-evolution: 8B proposer $\rightarrow$ 4B solver, compared with symmetric 4B $\rightarrow$ 4B from Table 1. Best per group in **green bold**.

Proposer	Setting	Pass@1 (%)						Avg.
		MATH500	GSM8K	AMC	Olympiad	AIME'24	AIME'25	
4B	Baseline	64.2 $\pm$ 0.8	85.2 $\pm$ 0.7	41.7 $\pm$ 1.4	23.2 $\pm$ 0.3	8.9 $\pm$ 0.9	6.7 $\pm$ 1.6	38.3
4B	+ VD85	65.7 $\pm$ 0.9	81.8 $\pm$ 0.4	50.0 $\pm$ 2.4	24.7 $\pm$ 0.9	8.9 $\pm$ 1.8	4.4 $\pm$ 0.9	39.3
8B	Baseline	63.5 $\pm$ 2.7	82.7 $\pm$ 2.5	44.2 $\pm$ 2.4	24.4 $\pm$ 0.1	11.1 $\pm$ 1.6	6.7 $\pm$ 5.4	38.8
8B	+ VD85 (gen-only)	66.1 $\pm$ 1.1	83.4 $\pm$ 2.7	35.8 $\pm$ 4.3	23.4 $\pm$ 0.7	13.3 $\pm$ 2.7	2.2 $\pm$ 1.6	37.4

The cross-scale baseline (8B $\rightarrow$ 4B, no dropout) performs comparably to the symmetric baseline (4B $\rightarrow$ 4B), at 38.8 vs. 38.3 avg. A stronger proposer does not automatically produce a better curriculum, likely because the self-consistency reward calibrates difficulty to the solver’s frontier regardless of proposer capacity.

Adding vocabulary dropout to the 8B proposer hurts rather than helps (37.4 vs. 38.8 avg), with a particularly large drop on AMC (−8.4) and AIME’25 (−4.5). This contrasts with

the symmetric setting, where VD85 improves the 4B solver by +1.0 avg. This is consistent with the scale-dependent findings in §6: vocabulary dropout addresses proposer stagnation during iterative co-adaptation, but when the proposer already has substantially more capacity than the solver, the difficulty calibration is misaligned and the extra diversity pressure makes the mismatch worse. This negative result supports the view that vocabulary dropout is a targeted intervention for co-evolutionary dynamics between capacity-matched models rather than a generic regularizer.

D.4 Instruction-tuned models

We use base (pre-trained) models throughout, consistent with R-Zero (Huang et al., 2026) and General-Reasoner (Ma et al., 2025). We additionally run the pipeline on Qwen2.5-1.5B-Instruct (Table 11). No configuration consistently improves over the untrained base on either benchmark. At the tested scale, co-evolutionary training, with or without vocabulary dropout, does not benefit instruction-tuned models in our experiments.

Table 11: Co-evolution on Qwen2.5-1.5B-Instruct (5 iterations).

Model	MATH500	AMC
Base (no training)	47.7	25.3
Baseline solver	47.3	26.0
VD75 solver	47.4	25.2

D.5 Vocabulary utilization

Table 12 reports the number of unique tokens used in generated questions at iteration 5. Despite having fewer tokens available per batch, VD75 proposers use substantially more of the vocabulary than the baseline. This is consistent with the non-stationary mask forcing the proposer to explore a wider range of token compositions rather than concentrating on a fixed subset.

Table 12: Unique tokens in iteration-5 generated questions.

Setting	Qwen3-4B		Qwen3-8B	
	Unique tok.	# Q	Unique tok.	# Q
Baseline	2,717	6,771	1,881	7,044
VD75	3,697	3,377	2,868	4,864
Δ	+36%		+52%	

D.6 Curriculum-transfer training details

This expands the curriculum-transfer experiment of §6.3. We take the iteration-5 question sets from the baseline, KL-Cov, Clip-Cov, and VD ($\alpha=0.75$) 8B runs, verify every problem with an external model (GPT-5-mini, greedy decoding, single attempt), and keep only those whose independently-derived answer matches the curriculum’s under `math_verify` equivalence. External verification keeps $\sim 20\text{--}31\%$ of each curriculum. We then GRPO-train Qwen3-4B-Base for 3 epochs on each verified subset using `prime-r1` (Intellect, 2025) (learning rate 1×10^{-6} , global batch size 32, group size 8, sequence length 2048, maximum completion length 1536, FSDP with DP=2 and a vLLM inference worker). Each checkpoint is evaluated `pass@1` under the protocol of Table 1, with Minerva Math added.

D.7 Annealed masking schedule

The scale-dependence of vocabulary dropout (Table 1) points toward adaptive masking schedules. We test a linear schedule increasing α from 0.75 to 1.0 over the 5 iterations (Figure 7). The anneal peaks two iterations earlier than fixed VD75 (iteration 3 vs. 5), wins 3

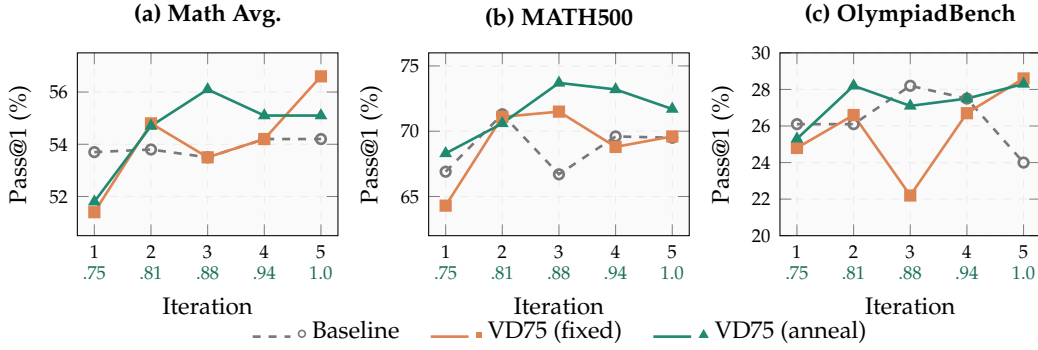


Figure 7: Qwen3-8B solver accuracy across iterations under fixed vs. annealed (0.75→1.0) vocabulary dropout. Teal α values below each tick show the anneal schedule. (a) Mean of MATH500, GSM8K, OlympiadBench, and Minerva Math.

of 5 iterations on math-average, and achieves the highest cumulative performance across all iterations. It also produces a more stable trajectory, with lower iteration-to-iteration volatility and an OlympiadBench range of 3.0 points vs. 6.4 for fixed (Figure 7c). These results suggest applying diversity pressure early, when stagnation risk is highest, then relaxing it as the solver matures. We leave this direction to future work.

D.8 Vocabulary dropout outside co-evolution

This section gives the setup and per-benchmark results behind the single-policy experiment in §6.4. We train Qwen3-4B-Base on GSM8K (Cobbe et al., 2021) with a binary correctness reward, with and without vocabulary dropout (a light mask, $\alpha=0.95$) applied to the policy during rollouts, and evaluate zero-shot on seven math benchmarks under the protocol of Table 1.

Table 13: Vocabulary dropout applied to a single GRPO policy, with no co-evolution: Qwen3-4B-Base trained on GSM8K with a binary correctness reward, with ($\alpha=0.95$) and without ($\alpha=1.0$) vocabulary dropout, evaluated zero-shot (pass@1 %). Best per benchmark in **green bold**.

Setting	MATH500	GSM8K	AMC	Olympiad	AIME'24	AIME'25	Minerva
Baseline ($\alpha=1.0$)	70.6	86.3	47.5	23.6	6.7	3.3	29.8
VD ($\alpha=0.95$)	63.0	87.9	45.0	24.1	10.0	0.0	34.9

Vocabulary dropout wins on four of the seven benchmarks (GSM8K, OlympiadBench, AIME 2024, Minerva), and the two arms are close on most tasks. The entropy trajectories for both arms are reported in Table 4.

E Hyperparameters

E.1 Evaluation benchmarks

Table 14: Evaluation benchmarks.

Benchmark	# Examples	Domain
GSM8K (Cobbe et al., 2021)	1,319	Grade-school math
MATH500 (Hendrycks et al., 2021)	500	Competition math
AMC (Mathematical Association of America, 2023)	40	Competition math
OlympiadBench (He et al., 2024)	910	Olympiad math
AIME 2024 (Mathematical Association of America, 2024)	30	Olympiad math
AIME 2025 (Mathematical Association of America, 2024)	30	Olympiad math
<i>Additional benchmarks (annealing experiments)</i>		
Minerva Math (Lewkowycz et al., 2022)	272	STEM math

E.2 Training hyperparameters

Table 15: GRPO training hyperparameters for proposer and solver.

Parameter	Proposer	Solver
Base model	Qwen3-4B / Qwen3-8B	
Optimizer	AdamW	
Learning rate	1×10^{-6}	
Weight decay	1×10^{-2}	
LR warmup	0	
Max grad norm	1.0	
KL penalty	low-variance KL, $\beta = 10^{-2}$	
Gradient checkpointing	enabled	
Precision	bfloat16	
Global batch size	16	8
Micro-batch (update)	2	2
Micro-batch (experience)	4	4
Rollout batch size	16	8
GRPO samples per prompt (G)	4	–
Self-consistency samples (M)	10	–
Temperature	1.0	1.0
Top- p	0.99	0.99
Max response length	2048	2048
Max steps per phase	6	20
Epochs per phase	10	10
Co-evolution iterations	5	
Difficulty window $[\tau_{\min}, \tau_{\max}]$	$[0.3, 0.7]$	
Vocab dropout α	0.75	–
BLEU rep. penalty τ_{BLEU}	0.5	–

E.3 Vendi score computation

We compute the Vendi score (Friedman & Dieng, 2023) for each (experiment, iteration) pair by embedding the proposer’s generated questions and computing the eigenspectrum of the cosine similarity kernel. Given n questions with L2-normalized embeddings $E \in \mathbb{R}^{n \times d}$, the similarity matrix is $K = EE^\top$. We compute $\text{VS} = \exp(-\sum_i \hat{\lambda}_i \log \hat{\lambda}_i)$ where $\hat{\lambda}_i = \lambda_i / \sum_j \lambda_j$ are the normalized eigenvalues of K . This yields the effective number of distinct question types.

Table 16: Vendi score hyperparameters.

Parameter	Value
Embedding model	text-embedding-3-small
Embedding dimension	1536
Normalization	L2 (cosine kernel)
Max samples (eigendecomp.)	5,000
Subsampling seed	42
Eigenvalue threshold	10^{-12}

E.4 Epiplexity computation

Our epiplexity computation follows [Finzi et al. \(2026\)](#), using the prequential MDL procedure that [Liu et al. \(2026\)](#) applied to self-play. For each (experiment, iteration) pair we fine-tune a fresh LoRA observer on the proposer’s generated questions and ask how much structure it can extract, defined as the gap between the online loss on each example before training and the converged training loss, in bits per token. A curriculum with more learnable structure yields higher epiplexity.

The observer starts from the same base model used in the co-evolutionary loop, with LoRA adapters on all attention and MLP projections. Training runs an online-then-converge protocol, where epoch 1 records the loss on each batch before updating (the prequential code length) and training then continues for up to 20 epochs with early stopping on a held-out validation set.

Table 17: Epiplexity computation hyperparameters.

Parameter	Value
Observer model	(same as base)
LoRA rank (r)	16
LoRA alpha	32
LoRA dropout	0.05
Target modules	q/k/v/o_proj, gate/up/down_proj
Max sequence length	512
Batch size	64
Learning rate	10^{-4}
Optimizer	AdamW (weight decay 0.01)
Max epochs	20
Early stopping patience	5 epochs
Validation split	10%
Min question length	20 characters
Precision	bfloat16

Epiplexity per token is computed as $(\mathcal{L}_{\text{online}} - \mathcal{L}_{\text{train}}) / \ln 2$, where $\mathcal{L}_{\text{online}}$ is the total cross-entropy loss from the prequential (epoch 1) pass and $\mathcal{L}_{\text{train}}$ is the total loss at the best MDL epoch. The MDL criterion selects the epoch that minimizes $\text{epiplexity} / N_{\text{train}} + \mathcal{L}_{\text{val}} / (\ln 2 \cdot N_{\text{val}})$.

F Prompt templates

F.1 Original R-Zero questioner prompt

The following is the out-of-the-box R-Zero questioner prompt, which targets various fields of mathematics at competition level. All results in §6 use this prompt.

R-Zero Questioner (System Message)

You are an expert competition-math problem setter. FIRST, in your private scratch-pad, think step-by-step to design a brand-new, non-trivial problem. The problem could come from any field of mathematics, including but not limited to algebra, geometry, number theory, combinatorics, prealgebra, probability, statistics, and calculus. Aim for a difficulty such that fewer than 30%

of advanced high-school students could solve it. Avoid re-using textbook clichés or famous contest problems.

THEN, without revealing any of your private thoughts, output **exactly** the following two blocks:

```
<question>
{The full problem statement on one or more lines}
</question>
```

```
\boxed{final_answer}
```

Do NOT output anything else: no explanations, no extra markup.

R-Zero Questioner (User Message)

Generate one new, challenging reasoning question now. Remember to format the output exactly as instructed.

F.2 Evaluation prompt

All benchmarks are evaluated zero-shot using the following user message, with no system prompt:

Evaluation (User Message)

```
{question}
Let's think step by step and provide your final answer inside \boxed{} notation.
```

The question text is passed verbatim from each dataset's problem field. The prompt is wrapped in the model's native chat template (e.g., `<|im_start|>user ... <|im_end|>` for Qwen3 models). No system message is set.